

CLASS NOTES · COMPLETE SERIES

OBJECTS, FAST

POP VS OOP · CLASSES AND OBJECTS · CONSTRUCTORS · OVERLOADING · INHERITANCE

Twelve chapters, each a screen or two, weighted the way the paper is weighted — this is a code paper, so the programs come first and the prose comes second. Every chapter closes with five lines that are the whole chapter, and there is a switch in the margin that hides everything except those lines.

That is the last-hour version.

12	56	9	63
CHAPTERS	SECTIONS	DIAGRAMS	SUMMARY LINES

CHAPTER 01

PROCEDURAL VERSUS OBJECT ORIENTED

This chapter settles the two or three marks that ask you to compare the two styles, and it settles the vocabulary used by every chapter after it. One sentence carries the whole idea: in a procedural program the data is out in the open and any function may change it, and in an object oriented program the data is locked inside an object with the functions that guard it.

1.1 How a procedural program is arranged

You write a list of instructions, wrap groups of them in functions, and the functions pass data between themselves. The design is **top-down**: start with the big job, break it into smaller jobs. Most of the data sits outside all the functions, where every function can reach it.

- Emphasis is on **doing things** — on the algorithm, on the sequence.
- A large program is split into smaller programs called **functions**.
- Most functions share **global data**, and data moves openly between them.
- A function transforms data from one form into another.

THE DRAWBACK, IN THE WORDS THE EXAMINER WANTS

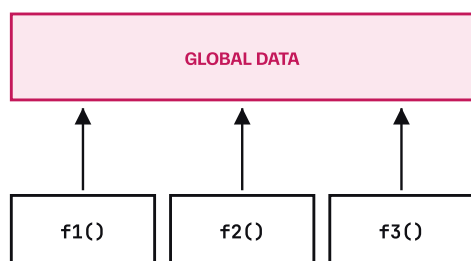
If asked for the main drawback of procedure oriented programming, the answer is **global data is exposed**. Data moves freely, any function can change it, so a bug in one function can break a completely unrelated part of the program. Object oriented programming exists to fix exactly this.

1.2 How an object oriented program is arranged

OOP turns the picture inside out. The data becomes the critical element and is not allowed to flow freely. A problem is broken into entities called **objects**, and the data of an object can be reached only by the functions belonging to that object. Objects talk to each other by calling each other's functions, never by reaching into each other's data.

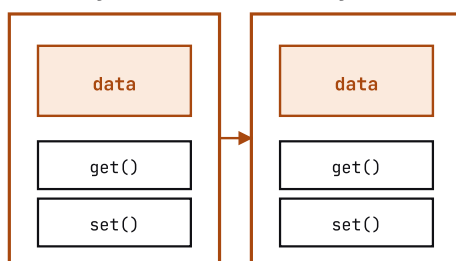
DIAGRAM

PROCEDURE ORIENTED



every function can reach the data.
change the data and you must check
every function that touches it.

OBJECT ORIENTED



the data is inside, and only the object's own
functions can reach it. Objects talk by
calling each other, not by sharing memory.

Fig. 1.1 The same program drawn twice. On the left, one block of global data with arrows running into it from every function — any arrow can change it. On the right, three closed boxes, each holding its data with its functions wrapped around it, and the only arrows between boxes are function calls.

1.3 The comparison, in the form to write it

QUESTION	PROCEDURE ORIENTED	OBJECT ORIENTED
Emphasis is on	Doing things — the algorithm	The data
Program is divided into	Functions	Objects
Where data lives	Global, shared by functions	Hidden inside the object
Who may change data	Any function that can see it	Only that object's own functions
Design direction	Top-down	Bottom-up
Adding new data	Means touching many functions	Added to one class, nothing else moves
Languages	C, Pascal, FORTRAN	C++, Java, C#

1.4 The six core ideas

- 1 Class** — a user-defined data type that acts as a blueprint. It wraps data members and the member functions that work on them into one unit.
- 2 Object** — an instance of a class, the actual thing built from the blueprint. Each object keeps its own copy of the data members.
- 3 Encapsulation** — wrapping data and functions into one unit and keeping the data hidden. In C++ the keyword that does it is `private`.
- 4 Abstraction** — showing only what the user needs and hiding the machinery. Encapsulation is *how* you hide; abstraction is *why*.
- 5 Inheritance** — one class takes on the properties of another. The giver is the base class, the receiver is the derived class. This is what gives OOP its reusability.
- 6 Polymorphism** — one name, many behaviours. Two forms in this course: function overloading and operator overloading.

1.5 The benefits, if they ask for four

- **Data security** — data is hidden, so outside functions cannot damage it by accident.
- **Reusability** — through inheritance, code written once is used in many classes.
- **Easy to extend** — new data and functions are added to a class without disturbing the rest of the program.
- **Manageable complexity** — a big problem becomes a set of objects, each small enough to understand alone.

- **Easier maintenance** — when a car misbehaves you look inside the `Car` class and nowhere else.

EXAM TIP

Every "distinguish between X and Y" question on this paper should be answered as a **table**, not as paragraphs. It is faster to write, faster to mark, and it stops you from repeating yourself. Four or five rows with a labelled first column is a full answer.

CHAPTER 01 AT A GLANCE

- The drawback of procedure oriented programming is that **global data is exposed** — any function can change it, so a bug anywhere can break anything.
- Object oriented programming locks the data inside the object and lets only that object's functions touch it.
- POP is top-down and organised around functions; OOP is bottom-up and organised around objects.
- The six core ideas: **class, object, encapsulation, abstraction, inheritance, polymorphism**.
- Encapsulation is the mechanism of hiding; abstraction is the purpose of hiding. That pair is asked constantly.

CHAPTER 02

TOKENS, TYPES AND REFERENCES

Two marks live here, and they are the cheapest two on the paper. This chapter settles the five kinds of token, the constants that are written in a way that catches people out, the difference between a structure, a union and a class, and the one derived type that C does not have at all — the reference.

2.1 The five kinds of token

The smallest individual units of a program are **tokens**. There are five: **keywords**, **identifiers**, **constants**, **strings**, **operators**. C++ has **63 keywords**, all in lower case, and none of them may be used as a name of your own.

LEARN THIS LINE

C++ Programs = Tokens + White Spaces + Syntax of the Programming Language + Logic. The keywords worth noticing are the new ones — `class` , `new` , `delete` , `friend` , `operator` , `public` , `private` , `this` , `virtual` — because they are exactly the object oriented features this course teaches.

2.2 The constants that catch people out

WRITTEN AS	WHAT IT ACTUALLY IS
123	Decimal integer
037	Octal — a leading zero means base 8, so this is 31 , not 37
0X2	Hexadecimal — a leading <code>0X</code> means base 16
'A'	One character, one byte
"A"	A string — one character plus a hidden <code>\0</code> , so two bytes
L'ab'	Wide-character constant — the <code>L</code> prefix

WATCH OUT

A character array in C++ must have room for the terminating null. `char s[3] = "xyz";` is legal C and illegal C++; C++ needs `char s[4] = "xyz";` . This exact pair of lines is a classic one-mark question.

2.3 Structure, union and class

	STRUCT	UNION	CLASS
Size	Sum of the sizes of its members	Size of the largest member	Sum, like a struct
Memory	Each member has its own storage	All members share one piece of storage	Each member has its own storage
Changing one member	Does not affect the others	Destroys the others' values	Does not affect the others
Members by default	public	public	private
Can hold functions	Yes, in C++	Yes, in C++	Yes

THE ONE-LINE ANSWER FOR STRUCT VERSUS CLASS

In C++ they are almost the same thing. The only real difference is that members of a `struct` are **public by default** and members of a `class` are **private by default**. The same default carries into inheritance: `struct D : B` is public inheritance, `class D : B` is private inheritance.

2.4 enum, and how it counts

```
enum shape { circle, square, triangle };    // 0, 1, 2
enum color { red, blue = 4, green };        // 0, 4, 5
enum color { red = 5, blue, green };        // 5, 6, 7
```

The rule is the exam trick: once you give one enumerator a value, the ones after it **carry on counting from there** unless they are given a value of their own.

2.5 The reference variable

A reference is an **alias** — a second name for a variable that already exists. It does not exist in C. Declare it with `&` in the declaration, and it must be initialised at that moment.

```
float total = 100;
float &sum = total;    // sum and total are ONE variable

int n[10];
int &x = n[8];         // x is another name for n[8]
```

THREE RULES THAT COST MARKS

1. A reference must be initialised when it is declared — `int &r;` alone is an error. **2.** Once bound it can never be re-pointed; `r = y;` copies the value of `y` into whatever `r` already refers to. **3.** Never return a reference to a local variable — it dies when the function ends.

PREDICT THE OUTPUT

Two small traps in one program: an assignment through a reference, and an enum that counts on from a value you gave it.

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int &r = a;          // r is an alias for a
    int b = 20;

    r = b;               // does NOT re-point r -- copies b's value into a
    cout << a << " " << r << " " << b << endl;

    enum color { red, blue = 4, green };
    cout << red << " " << blue << " " << green << endl;
    return 0;
}
```

Output

```
20 20 20
0 4 5
```

Line by line. `r` is bound to `a` for life, so `r = b;` is read as "put 20 into the thing `r` names", which is `a`. Now `a` and `r` are both 20, and `b` was never touched, so it is still 20 as well — three twenties, for two different reasons.

In the enum, `red` takes the default 0, `blue` is forced to 4, and `green` simply continues from `blue`, so it is 5. It is not 1, and it is not 2.

CHAPTER 02 AT A GLANCE

- Five kinds of token: **keywords, identifiers, constants, strings, operators**. C++ has **63** keywords.
- A leading zero means octal, so 037 is 31. 'A' is one byte; "A" is two, because of the hidden \0.
- A struct is public by default, a class is private by default — that is the only real difference between them in C++.
- In an enum, numbering continues from the last value you assigned, so {red, blue = 4, green} gives 0, 4, 5.
- A **reference** is an alias, must be initialised at declaration, and can never be made to refer to a different variable afterwards.

CHAPTER 03

THE OPERATORS C++ ADDS

Every C operator is still valid. On top of them C++ adds ten, and three of them are worth three marks on their own: the scope resolution operator, the new and delete pair, and the manipulators. This chapter settles what each one does and what happens when you use the wrong form of delete.

3.1 The ten new operators

OPERATOR	NAME	WHAT IT IS FOR
<<	Insertion	Sends data to an output stream
>>	Extraction	Reads data from an input stream
::	Scope resolution	Reaches a hidden global, or names a member outside its class
::*	Pointer to member declarator	Declares a pointer to a class member
->* and .*	Pointer to member	Uses such a pointer, through a pointer or through an object
new	Memory allocation	Takes memory from the free store
delete	Memory release	Gives it back
endl	Line feed	Ends the line and flushes
setw	Field width	Sets the width of the next item

3.2 The scope resolution operator, both of its jobs

The same operator does two completely different things, and the paper asks about both.

- 1

Unary `::name` — reach the *global* version of a name that a local declaration is hiding. In C a hidden global is simply unreachable; this is one of the small additions C++ makes.
- 2

Binary `Class::member` — name a member outside its class. This is what lets you define a member function outside the class body, define a static data member, call a static member function without an object, and disambiguate an inherited name.

```
int Box::area() { return len * len; }    // define a member function
int Item::count = 0;                    // define a static data member
Item::live();                           // call a static member function
p.M::display();                         // pick one of two inherited names
```

3.3 new and delete

```
int *p = new int;                       // one int, uninitialised
int *q = new int(25);                   // one int, initialised to 25
int *a = new int[10];                   // an array of ten ints

delete p;                               // one object
delete[] a;                             // an array -- note the brackets
```

	C – MALLOC / FREE	C++ – NEW / DELETE
What it is	A library function	An operator, built into the language
Needs a header	Yes, <code>stdlib.h</code>	No
Size	You calculate it: <code>malloc(sizeof(int))</code>	Worked out for you: <code>new int</code>
Return type	<code>void</code> pointer, usually needs a cast	Already the right pointer type
With an object	Does not call the constructor	Calls the constructor
Releasing	<code>free(p)</code> — no destructor runs	<code>delete p</code> — the destructor runs

THE THREE CLASSIC MEMORY ERRORS

1. Never calling `delete` — a memory leak. **2.** Writing `delete p;` for memory made with `new[]` — undefined behaviour: only the first element is destroyed and the wrong block is handed back, which usually corrupts the heap. With plain `int` it often seems to work, which makes it the worst kind of bug. **3.** Deleting the same pointer twice — also undefined, and usually a crash.

3.4 Manipulators

Manipulators format the output. Only `endl` comes free with `iostream`; `setw`, `setfill` and `setprecision` all need `iomanip`.

```
#include <iomanip>

cout << setw(10) << "Basic" << setw(10) << 10000 << endl;
cout << setw(20) << setfill('*') << "SVNIT" << endl;
cout << fixed << setprecision(2) << 3.141592653 << endl;

    Basic      10000
*****SVNIT
3.14
```

EXAM TIP

`setw` applies to the **next item only**, so it has to be repeated for every column.
`setfill` and `setprecision`, by contrast, stay in force until you change them.

PREDICT THE OUTPUT

The scope resolution operator and the two forms of `new` in one short program.

```
#include <iostream>
using namespace std;

int n = 5;                // global n

int main() {
    int n = 50;           // hides the global
    {
        int n = 500;      // hides main's n
        cout << n << " " << ::n << endl;
    }
    cout << n << " " << ::n << endl;

    int *p = new int(7);
    int *a = new int[3];
    a[0] = 1; a[1] = 2; a[2] = 3;
    cout << *p + a[2] << endl;

    delete p;
    delete[] a;
    return 0;
}
```

Output

```
500 5
50 5
10
```

Plain `n` always means the innermost declaration that is still in scope — 500 inside the block, 50 once the block has closed. `::n` ignores all of that and goes straight to file scope, so it is 5 both times, no matter how deep you are.

The last line is `7 + 3`. Note the two releases: `delete p;` for the single `int` and `delete[] a;` for the array. Swapping them is undefined behaviour, not a compile error, so the compiler will not warn you.

CHAPTER 03 AT A GLANCE

- Unary `::name` reaches the **global** version of a name that a local declaration is hiding — something C cannot do at all.
- Binary `Class::member` names a member outside its class: member function definitions, static data definitions, static calls, inherited-name disambiguation.
- `new` calls the constructor and returns the right pointer type; `malloc` does neither. `delete` calls the destructor; `free` does not.
- Whatever you take with `new[ ]`, give back with `delete[ ]`. Using plain `delete` on an array is **undefined behaviour**.
- `setw` affects only the next item and needs `ios manip`; `endl` is the only manipulator that comes free with `iostream`.

CHAPTER 04

OVERLOADING, INLINE AND DEFAULT ARGUMENTS

Function overloading is your first taste of polymorphism, and together with default arguments it is worth about five marks. This chapter settles what makes two functions different, the exact order the compiler searches in, why defaults belong in the prototype, and the one place where overloading and defaults collide.

4.1 What makes two functions different

Overloaded functions must differ in the **type, number or order** of their parameters. That set of things is called the signature.

CAN YOU OVERLOAD ON...	YES OR NO	WHY
A different number of parameters	Yes	The call itself shows how many you passed
Different parameter types	Yes	The compiler reads the types at the call
A different order of parameters	Yes	<code>(int, float)</code> and <code>(float, int)</code> are distinguishable
<code>const</code> on a pointer parameter	Yes	<code>char</code> pointer and <code>const char</code> pointer are genuinely different types
The return type only	No	The call does not reveal the return type
A <code>typedef</code> name	No	<code>typedef</code> renames a type, it does not create one

WATCH OUT

`int f(int);` and `double f(int);` are not overloads, they are a redeclaration conflict. Think of the statement `f(3);` written on its own line with the result thrown away — the compiler sees only the argument, so it has nothing to choose with.

4.2 The order the compiler searches in

OVERLOAD RESOLUTION, IN ORDER

Exact match → promotion → standard conversion → user-defined conversion → ellipsis. The compiler stops at the first level where **exactly one** candidate matches. If a level produces two candidates and there is no tie-break, the call is ambiguous — and the error appears at the call, not at the declarations.

The one that catches people: **promotion beats conversion**. Given `f(int)` and `f(double)`, a `char` argument goes to `f(int)`, because `char` to `int` is a promotion while `char` to `double` is only a conversion. A `float` argument goes the other way, to `f(double)`, for exactly the same reason.

4.3 Default arguments

```
int sum(int n1, int n2, int n3 = 0, int n4 = 0);    // defaults go HERE

int sum(int n1, int n2, int n3, int n4)           // NOT here
{ return n1 + n2 + n3 + n4; }
```

```
sum(10, 20);                // 30
sum(10, 20, 30);            // 60
sum(10, 20, 30, 40);        // 100
```

- 1 Default values go in the **prototype**, not in the definition. The compiler builds a call from the prototype.
- 2 Only **trailing** arguments may have defaults. A gap in the middle is illegal, because arguments are matched left to right and `sum(1, 2)` would become impossible to read.
- 3 The default may be a global constant, a global variable, or even a function call — `int f(int flag = display());` is legal.

EXAM TIP

The best reason to give for default arguments: they let you **add a parameter to a function without breaking any code that already calls it**. Old calls keep working with the old number of arguments.

4.4 Where overloading and defaults collide

```
void show(int a, int b = 10);    // can be called with ONE argument
void show(int a);                // can ALSO be called with one

show(5, 6);    // fine
show(5);       // ERROR -- ambiguous
```

A default argument does not create a second function. It lets the existing one be called with fewer arguments. So `show(5)` is an exact match for both declarations and neither is better. The fix is to remove the clash: delete one declaration, or drop the default.

4.5 Inline functions

Every call costs time: arguments are pushed on the stack, control is transferred, registers are saved, control comes back. For a two-line function that overhead can be most of the work. An **inline** function is expanded where it is called, so there is no call at all.

```
inline double cube(double a) { return a * a * a; }

c = cube(3.0);           // becomes 3.0 * 3.0 * 3.0
d = cube(2.5 + 1.5);     // correct -- a macro would get this WRONG
```

- `inline` is a **request, not a command**. The compiler may refuse, and usually does for a function with a loop, a switch, or recursion.
- A member function defined **inside** the class body is inline automatically — you do not write the keyword.
- The gain shrinks as the function grows, and each call site gets its own copy of the code, so a large inline function causes **code bloat**.
- It beats a macro because a macro is text substitution with no type checking: `#define Square(x) (x*x)` turns `Square(3+1)` into `3+1*3+1`, which is 7.

PREDICT THE OUTPUT

Default arguments and overload resolution in one go. Trace the two calls to `show` carefully — neither argument is an exact match for anything.

```
#include <iostream>
using namespace std;

int sum(int a, int b, int c = 100);           // default in the prototype

void show(int x)    { cout << "int "    << x << endl; }
void show(double x) { cout << "double " << x << endl; }

int main() {
    cout << sum(1, 2) << endl;
    cout << sum(1, 2, 3) << endl;
    show('A');
    show(2.5f);
    return 0;
}

int sum(int a, int b, int c) { return a + b + c; } // no default here
```

Output

```
103
6
int 65
double 2.5
```

`sum(1, 2)` takes the default, so it is $1 + 2 + 100$. `sum(1, 2, 3)` supplies its own third argument, so the default is never used.

`show('A')` has no exact match. `char` to `int` is an **integral promotion**; `char` to `double` is only a standard conversion. Promotion ranks higher, so `show(int)` wins and prints the code for 'A', which is **65**.

`show(2.5f)` is the same rule running the other way: `float` to `double` is a promotion, `float` to `int` is a lossy conversion. So `show(double)` wins and prints **2.5**.

CHAPTER 04 AT A GLANCE

- Overloaded functions must differ in the **type, number or order** of parameters. A different return type alone is not an overload, it is an error.
- Resolution order: exact match, then promotion, then standard conversion, then user-defined conversion, then ellipsis — and **promotion beats conversion**.
- Default argument values belong in the **prototype**, and only trailing arguments may have them.
- A one-argument call that matches both an overload and a defaulted parameter is **ambiguous**, and the error appears at the call, not at the declaration.
- `inline` is a request the compiler may refuse; a member function defined inside the class body is inline automatically.

CHAPTER 05

CALL BY VALUE, ADDRESS AND REFERENCE

Three marks, almost guaranteed, and the classic question is the swap program written all three ways. This chapter settles what is actually passed in each case, what the call site looks like, and which one to choose — plus the one thing that makes call by reference slightly dangerous.

5.1 The three mechanisms

```
void swap(int a, int b)      // by VALUE -- a and b are copies
{ int t = a; a = b; b = t; }
swap(x, y);                 // x and y are unchanged

void swap(int *a, int *b)   // by ADDRESS -- a and b hold addresses
{ int t = *a; *a = *b; *b = t; }
swap(&x, &y);               // the & at the call gives it away

void swap(int &a, int &b)    // by REFERENCE -- a and b are aliases
{ int t = a; a = b; b = t; }
swap(x, y);                // looks exactly like call by value
```

5.2 The table to write in the exam

	CALL BY VALUE	CALL BY ADDRESS	CALL BY REFERENCE
What is passed	A copy of the value	The address, itself passed by value	An alias for the original
Header looks like	<code>f(int a)</code>	<code>f(int *a)</code>	<code>f(int &a)</code>
Call looks like	<code>f(x);</code>	<code>f(&x);</code>	<code>f(x);</code>
Inside, you write	<code>a</code>	<code>*a</code> , every single time	<code>a</code>
Changes the original	No	Yes	Yes
Can be null	Not applicable	Yes, so it must be checked	No — a reference always refers to something
Available in	C and C++	C and C++	C++ only

5.3 Which to use, and why

- **Value** — for small built-in types you do not intend to modify. It is the safest, because the function physically cannot damage the caller's data.
- **Reference** — when the function must modify the argument, and for large objects you want to pass without copying. Add `const` when you want the speed of a reference with the safety of a value.
- **Address** — when the argument is genuinely optional, so that a null pointer means something, or when interfacing with C.

THE ONE REAL DOWNSIDE OF CALL BY REFERENCE

The call site is identical to call by value. A reader of `f(x);` cannot tell that `x` is about to be modified — only the function header says so. Call by address at least announces itself with the `&` at the call.

PREDICT THE OUTPUT

Three functions, three mechanisms, one line of output.

```
#include <iostream>
using namespace std;

void f1(int x) { x = x * 2; }
void f2(int *x) { *x = *x * 2; }
void f3(int &x) { x = x * 2; }

int main() {
    int a = 3, b = 3, c = 3;
    f1(a);
    f2(&b);
    f3(c);
    cout << a << " " << b << " " << c << endl;
    return 0;
}
```

Output

```
3 6 6
```

`f1` doubled its own private copy and then threw it away, so `a` is still 3. `f2` received the address of `b` and wrote through it, so `b` is 6. `f3` received an alias for `c`, so writing to `x` is writing to `c`, and it is 6 as well.

The exam point is the pair of call sites `f1(a)` and `f3(c)`. They look identical, yet one is harmless and the other modifies the caller. References are a C++ addition — C has only value and address — and this is the price of the convenience.

CHAPTER 05 AT A GLANCE

- Call by value passes a **copy**, so the caller's variable can never change.
- Call by address passes the **address**; inside the function you must dereference with a star every time, and the call site shows an &.
- Call by reference passes an **alias**; the function body looks like call by value and so does the call, but the original does change.
- Call by reference is the only one of the three that C does not have.
- Use value for small unmodified types, reference to modify or to avoid copying a large object, address when a null argument has to be meaningful.

CHAPTER 06

CLASSES AND OBJECTS

This is the subject itself, and four or five marks of the paper. The chapter settles how a class is declared, what private really means, the two places a member function body may live, and what memory actually looks like when you make ten objects. It ends with the find-the-errors question in the shape the paper sets it.

6.1 Declaring a class

```
class item
{
    int number;           // private by default
    float cost;

public:
    void getdata(int a, float b);
    void putdata(void);
};                        // the semicolon is part of the syntax

item x;                  // an object
item arr[10];            // an array of objects
```

THE MOST COMMON SYNTAX ERROR IN THE COURSE

A class declaration ends with `};` and not with `}`. Leave the semicolon out and the compiler reports a wall of errors that all point at the **next** line, not at the class.

6.2 private, public, protected

SPECIFIER	ITS OWN CLASS	A DERIVED CLASS	THE OUTSIDE WORLD
private	yes	no	no
protected	yes	yes	no
public	yes	yes	yes

Members are **private by default** in a class. An object may call a public member function, but it may not touch a private data member: `x.getdata(100, 75.5);` is legal and `x.number = 100;` is not. That is encapsulation actually working — and it is the planted error in nearly every find-the-errors question.

STATE IT IN EXACTLY THESE WORDS

Objects cannot access the private members of a class directly. They must go through a public member function — which can check the value first.

6.3 Where the member function body goes

```
// OUTSIDE the class -- the usual form
void item :: getdata(int a, float b)
{ number = a; cost = b; }

// INSIDE the class -- automatically inline
class item {
    int number;
public:
    void putdata(void) { cout << number << endl; }
};
```

The membership label `item ::` is what tells the compiler the function belongs to `item`, and that is what gives it permission to touch the private members. Leave it out and you have written an ordinary global function that cannot see them. Three facts follow: several classes may use the same function name, a member function may reach the private data, and a member function may call another member function of the same class with no dot at all.

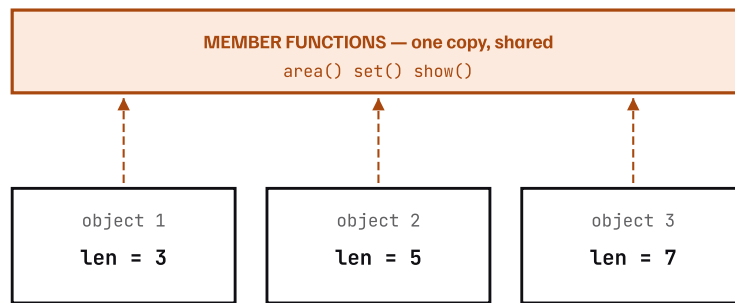
6.4 What memory looks like

ONE SENTENCE, ONE WHOLE ANSWER

Memory for **data members** is **allocated separately for each object**; **member functions are stored once** and shared by every object of the class. Ten objects means ten sets of data and one copy of the code.

DIAGRAM

WHAT AN OBJECT ACTUALLY COSTS



every object gets its own copy of the DATA members – that is all `sizeof(obj)` measures.
 the FUNCTIONS exist once for the whole class. Three objects do not mean three copies of `area()`.
 the `this` pointer is how one shared function knows which object it was called on.

Fig. 6.1 One wide box at the top labelled with the member functions, marked "one copy, shared". Below it, three small identical boxes, one per object, each holding its own `number` and `cost`, with dotted arrows from all three up to the shared code.

6.5 Objects as arguments and return values

```

class complex {
    float x, y;
public:
    void input(float r, float i) { x = r; y = i; }

    complex sum(complex c)           // object in, object out
    {
        complex temp;
        temp.x = x + c.x;           // c.x is PRIVATE -- and legal
        temp.y = y + c.y;
        return temp;
    }
    void show() { cout << x << " + j" << y << endl; }
};

c3 = c1.sum(c2);                    // 2.5+j3.5 plus 1.6+j2.7 → 4.1 + j6.2
  
```

THE SURPRISING RULE

Privacy in C++ is **per class, not per object**. A member function of `complex` may read the private members of any `complex`, not only its own. That is why `temp.x` and `c.x` are both allowed inside `sum`. Remember this program — in Chapter 10 it becomes `c3 = c1 + c2;`.

FIND THE FIVE ERRORS

The program below does not compile. Give the line, say why, and write the correction — that is exactly how the marks are split, one per error, half for naming a line without fixing it.

```
1  #include <iostream>
2  using namespace std;
3
4  class Box {
5      int len;
6  public:
7      Box(int l) { len = l; }
8      int area();
9  }
10
11 int area() { return len * len; }
12
13 int main() {
14     Box b;
15     b.len = 5;
16     cout << b.area;
17     return 0;
18 }
```

Line 9 — a class declaration must end with `};`. Add the semicolon.

Line 11 — the function is defined without the class name, so it is a global function, not `Box`'s member, and a global function cannot see `len`. Two errors for the price of one. Write `int Box::area() { return len * len; }`.

Line 14 — `Box b;` calls a default constructor, but the moment you write any constructor of your own the compiler stops supplying one. Write `Box b(5);`, or add `Box() { len = 0; }` to the class.

Line 15 — `len` is private and `main()` is outside the class. Delete the line, or add a public `void set(int l) { len = l; }` and call `b.set(5);`.

Line 16 — `b.area` is the function itself, not a call. A member function name without brackets cannot be streamed. Write `cout << b.area() << endl;`.

Corrected, the program prints **25**.

CHAPTER 06 AT A GLANCE

- A class declaration ends with `};` — the missing semicolon is the single most common syntax error in this subject.
- Members of a class are **private by default**, and an object may reach them only through a public member function.
- A member function defined outside the class needs the membership label `ClassName::`; without it, it is a global function with no access to private data.
- A member function defined **inside** the class body is inline automatically.
- Data members get one copy **per object**; member functions get **one copy for the whole class**.
- Privacy is per class, not per object — a member function may read the private data of any object of its own class.

CHAPTER 07

STATIC MEMBERS, FRIENDS, CONST AND THIS

Six marks are spread across this chapter: a static question and a friend question are both very likely. What they have in common is that each one breaks a rule you were taught earlier — static data belongs to no object, a friend is not a member, a const function may not write, and `this` is the hidden argument that explains all of it.

7.1 static data members

- 1 Only **one copy** exists for the entire class, shared by every object, no matter how many objects are created.
- 2 It is initialised to **zero** when the first object is created. No other initialisation is permitted.
- 3 It is visible only within the class, but its lifetime is the whole program.
- 4 It must be **defined outside the class** to give it storage.

```
class item {
    static int count;      // declared -- says it exists
    int number;
public:
    void getdata(int a) { number = a; count++; }
    static int live() { return count; }
};

int item :: count = 0;    // DEFINED -- storage happens here
```

THE LINE EVERYBODY FORGETS

`int item::count = 0;` at file scope is not optional. Without it the program compiles and then fails at **link** time with "undefined reference to `item::count`". Note that the definition does not repeat the word `static`, and that it is allowed to touch a private member because it is part of the class.

7.2 static member functions

- It can be called using the **class name**, with no object at all: `item::live();`
- It has **no `this` pointer**, and therefore it may only touch other static members.
- The reason is simple: it can be called before any object exists, so if it tried to read `number` the compiler would have to ask "whose number?" and there would be no answer.

7.3 friend functions

```

class B;                                // forward declaration -- REQUIRED

class A {
    int x;
public:
    A(int v) { x = v; }
    friend int sum(A, B);                // B is only a NAME here, and that is enough
};

class B {
    int y;
public:
    B(int v) { y = v; }
    friend int sum(A, B);
};

int sum(A a, B b) { return a.x + b.y; } // no "friend", no "A::"

```

The forward declaration is needed because class `A` is compiled first and its friend declaration mentions `B`, a name the compiler has not met. `class B;` promises that a class of that name exists, which is all the compiler needs to parse the parameter list.

THE PROPERTIES OF A FRIEND FUNCTION – THE LIST THEY WANT

1. It is not a member of the class, even though it is declared inside it, and it is not in the class scope. **2.** It is called like an ordinary function, with no object: `sum(a, b)`, never `a.sum(b)`. **3.** It has **no this pointer**, so every object it works on must be passed as an argument — which is why it takes one more argument than the member version would. **4.** It may be declared in the public or the private section; it makes no difference. **5.** It can reach both private and protected members. **6.** Friendship is **granted, not taken**; it is **not mutual**; and it is **not inherited**.

WHY C++ ALLOWS IT AT ALL

It looks like a hole in encapsulation and it is — a deliberate one. Two situations need it: a function that must reach inside two unrelated classes, and an operator whose left operand is not an object of your class. The class still controls who gets in, because the invitation is written in the class itself.

7.4 const member functions

```
class item {  
    int number;  
public:  
    int getnumber() const;    // const goes at the END  
};  
  
int item :: getnumber() const    // and again in the definition  
{ return number; }            // writing number = 5; here is a compile error
```

Mark every getter and every display function `const`. It documents the promise, the compiler enforces it, and — the practical reason — a `const` object can only call `const` member functions. Forget the `const` and your function becomes unusable on const objects and on const references.

7.5 The this pointer

Every non-static member function receives a hidden extra argument: a pointer to the object it was called on, named `this`. When you write `number` inside a member function, the compiler reads it as `this->number`. Three places where you have to write it yourself:

- To return the object itself, so that chained assignment works: `return *this;`
- To guard against self-assignment: `if (this != &other) { ... }`
- When a parameter has the same name as a data member.

It also explains two rules you already know. A **static** member function has no `this`, which is why it cannot touch ordinary data members. A **friend** function has no `this` either, which is why it needs every object handed to it as an argument.

PREDICT THE OUTPUT

One shared counter, three separate ids, and a call with no object.

```
#include <iostream>
using namespace std;

class Item {
    static int count;
    int id;
public:
    void set() { id = ++count; }
    void show() { cout << "id " << id << ", count " << count << endl; }
    static int live() { return count; }
};

int Item::count = 0;           // definition at file scope

int main() {
    Item a, b, c;
    a.set(); b.set(); c.set();
    a.show();
    c.show();
    cout << Item::live() << endl;
    return 0;
}
```

Output

```
id1, count 3
id3, count 3
3
```

`count` is not one variable per object — it is a single variable shared by every `Item`. That is why the ids come out 1, 2, 3 instead of every object thinking it is number 1, and why `a` reports a count of 3 even though `a` itself was the first one made.

`live()` is `static`, so it is called as `Item::live()` with no object at all. It is allowed to read `count` because `count` is static too; it could not read `id`, because there would be no object to read it from.

CHAPTER 07 AT A GLANCE

- A static data member is **one variable shared by the whole class**, and it must be defined once outside the class or the program will not link.
- A static member function is called as `ClassName::f()`, has no `this`, and may touch only other static members.
- A **friend function is not a member**: no dot, no `ClassName::`, no `this`, and every object it works on is passed as an argument.
- Friendship is granted by the class, is not mutual, and is not inherited.
- A const member function may not modify the object, and a const object can call nothing else.
- `this` is the hidden pointer to the calling object; return `*this`; and if `(this != &other)` are the two places you write it.

CHAPTER 08

CONSTRUCTORS AND DESTRUCTORS

Four or five marks, and the single most likely predict-the-output question on the paper. This chapter settles the four rules a constructor obeys, the four kinds of constructor, the trap where the free default constructor disappears, and the ordering rule that every trace question is really testing.

8.1 The four rules

- 1 It has the **same name as the class**.
- 2 It is called **automatically** when an object is created. You never call it yourself.
- 3 It has **no return type** — not even `void`.
- 4 It should be declared in the **public** section.

Why they exist: without a constructor a freshly made object holds whatever rubbish was in that memory. A constructor guarantees an object is never in an invalid state, not even for one instruction.

8.2 The four kinds

```
class integer {
    int m, n;
public:
    integer()          { m = 0; n = 0; }      // 1 default
    integer(int a, int b){ m = a; n = b; }      // 2 parameterized
    integer(integer &i) { m = i.m; n = i.n; }    // 3 copy
};

integer I1;           // default
integer I2(20, 40);    // parameterized, implicit
integer I3 = integer(0,10); // parameterized, explicit
integer I4(I2);        // copy
```

The fourth kind is the **dynamic constructor** — one that calls `new` while the object is being built, so the object decides at run time how much memory it needs. That is the `String` class of Chapter 9, and whatever it takes with `new` the destructor must give back with `delete`.

8.3 The default constructor that disappears

THE PLANTED ERROR IN EVERY FIND-THE-ERRORS QUESTION

The compiler supplies a free default constructor **only while you have written none of your own**. Write `Box(int l)` and `Box b;` becomes a compile error. If you need both forms, write both constructors.

A second, quieter trap: if a class has both `complex()` and `complex(float r = 0, float i = 0)`, then `complex c;` is **ambiguous**, because both can be called with zero arguments. Do not write both.

8.4 Destructors

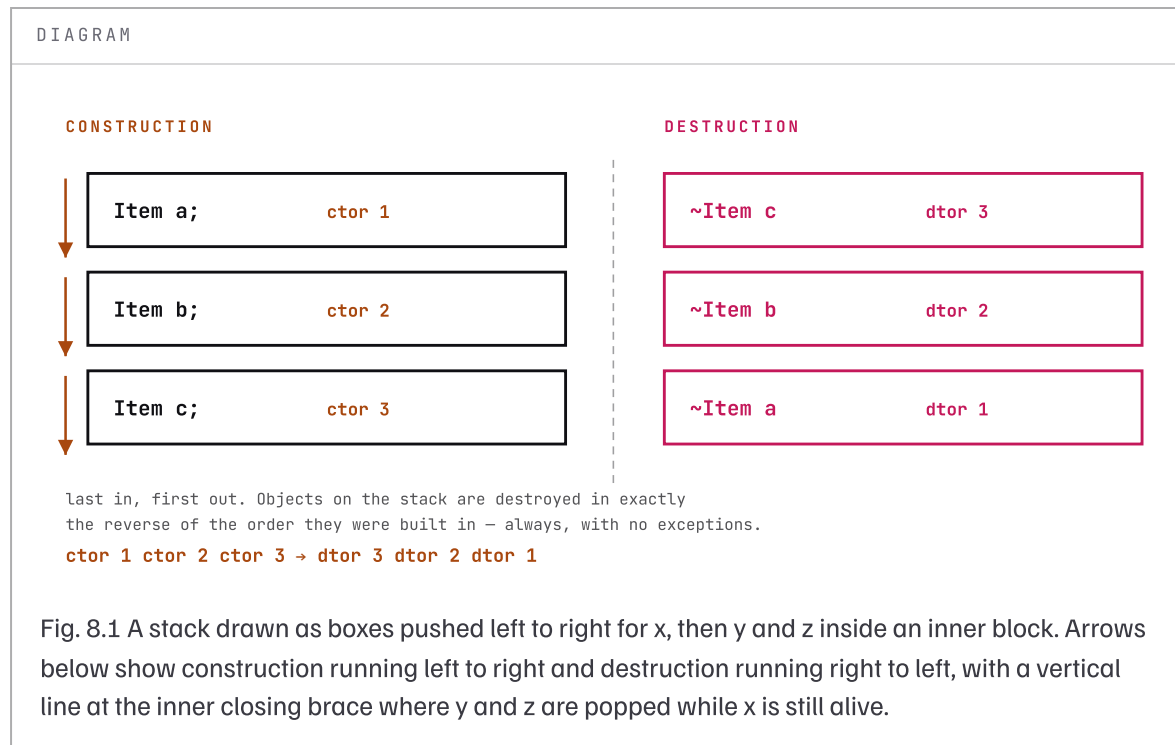
- Same name as the class with a tilde in front: `~item()`.
- Takes **no arguments** and returns nothing.
- Called **automatically** when the object goes out of scope.
- There is exactly **one per class** — a destructor cannot be overloaded.
- Its real job is releasing what the constructor took: `delete[] name;`

	CONSTRUCTOR	DESTRUCTOR
Name	Same as the class	Same as the class with <code>~</code> in front
When it runs	When an object is created	When an object is destroyed
Arguments	Can take them	Never
Return type	None	None
Overloading	Can be overloaded	Cannot — one per class
Typical job	Initialise data, allocate memory	Release memory, close files
Order for several objects	Order of creation	Reverse order of creation

8.5 The ordering rule

THE RULE EVERY TRACE QUESTION IS TESTING

Objects are destroyed in the **exact reverse** of the order they were created, and destruction follows **scope**: an object declared inside an inner block dies at that block's closing brace, not at the end of `main`.



PREDICT THE OUTPUT

Eight lines of output, and the order is the whole question. Trace it with a finger on the closing brace.

```
#include <iostream>
using namespace std;

class A {
    int n;
public:
    A(int k) { n = k; cout << "ctor " << n << endl; }
    ~A()      {          cout << "dtor " << n << endl; }
};

int main() {
    A x(1);
    {
        A y(2), z(3);
        cout << "-- leaving block --" << endl;
    }
    cout << "-- back in main --" << endl;
    return 0;
}
```

Output

```
ctor 1
ctor 2
ctor 3
-- leaving block --
dtor 3
dtor 2
-- back in main --
dtor 1
```

Two things produce that order. First, objects on the stack are destroyed last in, first out, so inside the block it is `z` before `y` — you see `dtor 3` before `dtor 2`, never the other way round.

Second, the inner block has its own scope, so `y` and `z` die at the closing brace, while `x` is still alive and keeps going until `main` ends. That is why `dtor 1` is last and appears after the final message, not before it.

CHAPTER 08 AT A GLANCE

- A constructor has the class's name, no return type, runs automatically, and belongs in the public section.
- Four kinds: **default, parameterized, copy, dynamic**. Constructors can be overloaded; a destructor cannot.
- Write any constructor of your own and the free default one disappears — `Box b;` then fails to compile.
- A destructor is `~ClassName()`, takes no arguments, and releases whatever the constructor allocated.
- Objects are destroyed in the **reverse order of creation**, and an object in an inner block dies at that block's closing brace.

CHAPTER 09

THE COPY CONSTRUCTOR, SHALLOW AND DEEP

Three or four marks, and the question is almost always the same one: a class holding a raw pointer, no copy constructor, and two objects that end up sharing one buffer. This chapter settles when the copy constructor runs, why its parameter must be a reference, what a shallow copy breaks, and the rule of three that fixes it.

9.1 When the copy constructor runs

```
integer I2(20, 40);

integer I3(I2);      // 1. explicit copy
integer I4 = I2;     // 2. copy initialisation -- SAME thing

integer I5;
I5 = I2;             // NOT the copy constructor -- assignment
```

- When an object is used to build another: `I3(I2)` and `I4 = I2`.
- When an object is passed to a function **by value** — the function's copy is built by the copy constructor.
- When an object is returned from a function **by value**.

THE DISTINCTION PEOPLE GET WRONG

`integer I4 = I2;` calls the **copy constructor**, because `I4` is being created. `I4 = I2;` on a later line, when `I4` already exists, calls the **assignment operator**. Same symbol, two different jobs, decided by whether the object already exists.

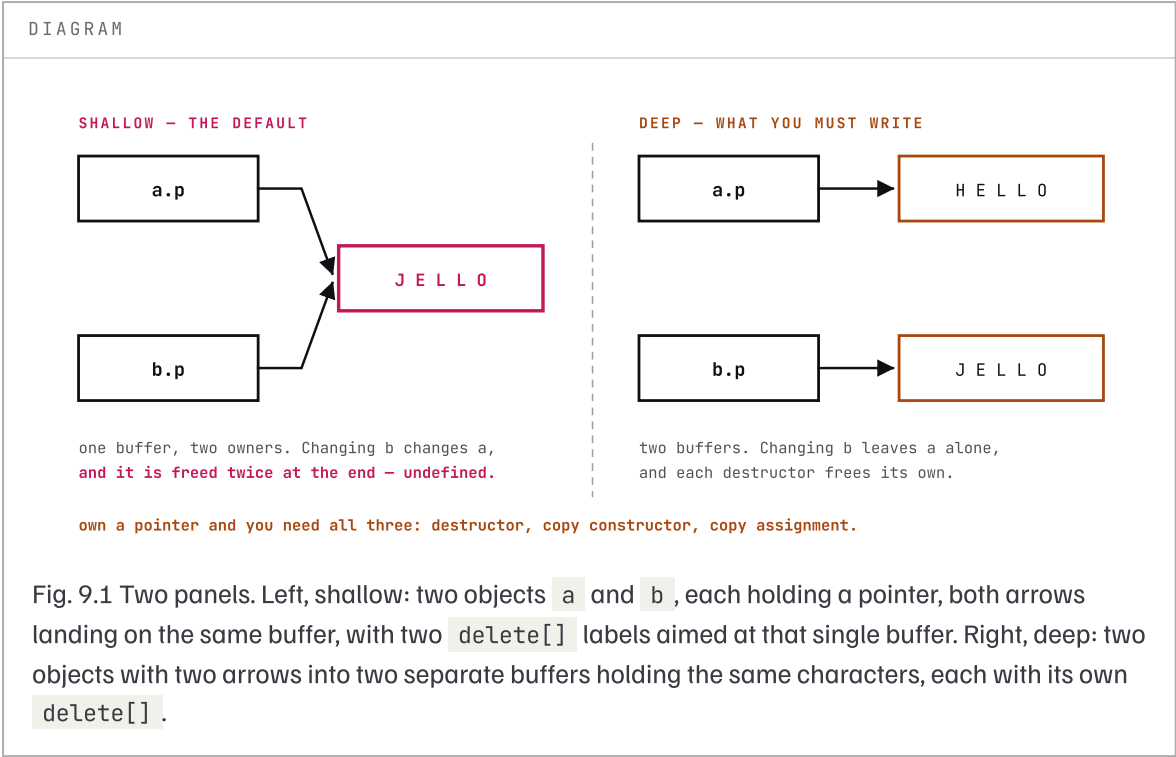
9.2 Why the parameter must be a reference

```
class A { public: A(A); };    // ERROR -- by value
class A { public: A(A&); };  // correct -- by reference
```

Passing an object by value means making a copy. Making a copy means calling the copy constructor. So `A(A)` would have to call itself to build its own argument, which would call itself again, forever. The compiler rejects it outright. Write it as `A(const A &a)` — the `const` is good practice, the `&` is compulsory.

9.3 What a shallow copy breaks

If you write no copy constructor, the compiler supplies one, and it does a plain **member by member copy**. When the only member is a pointer, what gets copied is the address — not the thing it points at.



	SHALLOW COPY	DEEP COPY
Who writes it	The compiler, if you write nothing	You, by hand
What is copied	The pointer — the address	A fresh buffer with the same contents
After the copy	Two objects share one buffer	Two independent objects
Changing one	Silently changes the other	Changes only that one
At destruction	The same block is released twice — undefined behaviour	Each object releases its own block
Needed when	Never, if the class owns a pointer	Always, if the class owns a pointer

9.4 The fix, in full

```
class Str {
    char *p;
public:
    Str(const char *s) { p = new char[strlen(s)+1]; strcpy(p, s); }

    Str(const Str &s)           // DEEP copy constructor
    {
        p = new char[strlen(s.p)+1];
        strcpy(p, s.p);
    }

    Str& operator=(const Str &s) // or "a = b;" has the same bug
    {
        if (this != &s) {      // guard against a = a;
            delete[] p;
            p = new char[strlen(s.p)+1];
            strcpy(p, s.p);
        }
        return *this;          // so that a = b = c; works
    }

    ~Str() { delete[] p; }
};
```

THE RULE OF THREE, IN THE FORM TO QUOTE IT

The moment a class owns a resource through a raw pointer it needs all three of a **destructor**, a **copy constructor** and a **copy assignment operator**. Write one and you almost certainly need the other two.

PREDICT THE OUTPUT

Two defects, one output. Say what is printed and then name both defects — that is how the question is set.

```
#include <iostream>
#include <cstring>
using namespace std;

class Str {
    char *p;
public:
    Str(const char *s) { p = new char[strlen(s)+1]; strcpy(p, s); }
    ~Str()              { delete[] p; }
    void set(int i, char c) { p[i] = c; }
    void show()           { cout << p << endl; }
};

int main() {
    Str a("HELLO");
    Str b = a;          // no copy constructor written
    b.set(0, 'J');
    a.show();
    b.show();
    return 0;
}
```

Output

```
JELLO
JELLO
```

...and then, on the way out of `main`, the program has **undefined behaviour**. Say that plainly rather than inventing a result: in practice it is a heap corruption or a crash, but the language does not define what happens.

Defect 1, aliasing. No copy constructor was written, so the compiler supplied one, and it copied the single member `p` — the address. Both objects now point at the one buffer, so `b.set(0, 'J')` writes through `b.p` and `a` reads the same bytes. Two objects that were meant to be independent are not.

Defect 2, the double delete. At the end of `main`, `b` is destroyed and runs `delete[] p`. Then `a` is destroyed and runs `delete[]` on the same address. This is the more serious of the two, because it can strike long after the real mistake.

The fix is a deep copy constructor plus a matching assignment operator, as in section 9.4. With them, the output becomes `HELLO` then `JELLO`, and nothing crashes.

CHAPTER 09 AT A GLANCE

- The copy constructor runs when an object is **created** from another, and when an object is passed or returned **by value**.
- `X b = a;` is the copy constructor because `b` is being created; `b = a;` on a later line is assignment.
- The parameter must be a reference — `A(A)` would need a copy to build its own argument, forever.
- A compiler-supplied copy is **shallow**: it copies the pointer, so two objects end up sharing one buffer and deleting it twice.
- The **rule of three**: a class that owns memory through a raw pointer needs a destructor, a copy constructor and a copy assignment operator.

CHAPTER 10

OPERATOR OVERLOADING, MEMBER OR FRIEND

Four marks, usually a Complex class you have to write and then a "why does this not work" follow-up. Everything in this chapter falls out of one fact: the compiler rewrites every operator as a function call, and which rewrite it picks decides what is possible.

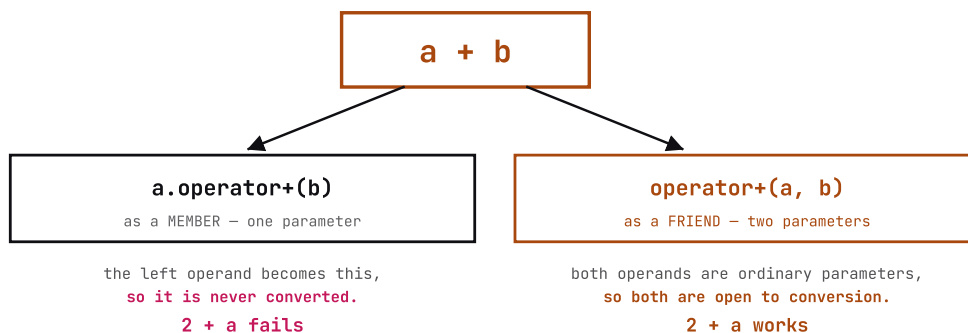
10.1 What the compiler actually does

```
c3 = c1 + c2;
```

```
as a MEMBER    →   c3 = c1.operator+(c2);    // left operand is the object
```

```
as a FRIEND    →   c3 = operator+(c1, c2);    // both operands are arguments
```

DIAGRAM



if the left operand might not be your class, it has to be a friend – which is why << always is.

Fig. 10.1 The expression `a + b` with two arrows leaving it. One leads to `a.operator+(b)` with `a` labelled "the calling object, supplied as this" and `b` labelled "the one argument". The other leads to `operator+(a, b)` with both operands labelled "ordinary argument, open to conversion".

THE ONE SENTENCE EVERYTHING FOLLOWS FROM

For a **member** operator, `a + b` becomes `a.operator+(b)` : the left operand becomes the calling object and the right one becomes the argument. So a binary member operator takes **one** parameter and a binary friend takes **two**.

10.2 Writing one

```
class Complex {
    float re, im;
public:
    Complex()                { re = im = 0; }
    Complex(float r, float i) { re = r; im = i; }

    Complex operator+(Complex c)    // binary, member: ONE argument
    {
        Complex t;
        t.re = re + c.re;
        t.im = im + c.im;
        return t;
    }

    void operator-()            // unary, member: NO arguments
    { re = -re; im = -im; }

    void show() { cout << re << " + j" << im << endl; }
};
```

OPERATOR KIND	AS A MEMBER FUNCTION	AS A FRIEND FUNCTION
Unary — one operand	0 arguments	1 argument
Binary — two operands	1 argument	2 arguments

A friend always needs one more argument than the member version, because it has no calling object to supply the left operand.

10.3 Prefix and postfix

```
void operator++()    { ... }    // prefix    ++x
void operator++(int) { ... }    // postfix    x++
```

The `int` is never used and has no name. It exists purely as a signal to the compiler meaning "this is the postfix version". Odd syntax, but it is the standard way and it is examinable.

10.4 Why `2 + a` fails, and how a friend fixes it

Suppose `Complex` has a one-argument constructor `Complex(float)`. Then `c = a + 2;` works: the compiler sees `a.operator+(2)`, the parameter wants a `Complex`, and a one-argument constructor is a **converting constructor**, whether you meant it to be or not.

```
c = a + 2;      // becomes a.operator+(2) -- fine, 2 converts to Complex
c = 2 + a;      // would become 2.operator+(a) -- NONSENSE
```

The left operand is an `int`, and an `int` is a built-in type with no member functions. The compiler will never convert the left operand of a member operator, because it has to find the function before it can consider conversions, and it looks for it inside the type of the left operand. Declare the operator a **friend** and both operands become ordinary parameters, so both are open to conversion and all three forms compile.

USE A MEMBER FUNCTION WHEN

The left operand is always an object of your class

The operator modifies the object: `=`, `+=`, `++`

The language requires it: `=`, `()`, `[]`, `->`

USE A FRIEND FUNCTION WHEN

The left operand may be a built-in type, as in `2 + a`

You want both operand orders to work

The operation is symmetric and belongs to neither side

THE ONE-LINE RULE

If the left operand may be something other than your class, it must be a friend. That is why `operator<<` and `operator>>` are always friends — their left operand is a stream, not your object.

10.5 The two lists to memorise

TWO DIFFERENT LISTS, BOTH ASKED

Cannot be overloaded **at all**: `::`, `.`, `.*`, `?:` — four of them. Must be a **member**, never a friend: `=`, `()`, `[]`, `->` — also four. Do not mix the lists up.

- Only existing operators can be overloaded — you cannot invent a symbol.
- At least one operand must be a user-defined type.
- You cannot change an operator's basic meaning, its precedence, its associativity, or the number of operands it takes.

10.6 A string class, which is the whole syllabus in forty lines

```
class String {
    char *p;
    int len;
public:
    String()          { len = 0; p = new char[1]; p[0] = '\\0'; }
    String(const char *s) { len = strlen(s); p = new char[len+1]; strcpy(p,s); }
    String(const String &s) { len = s.len;    p = new char[len+1]; strcpy(p,s.p); }
    ~String()          { delete[] p; }

    friend String operator+(const String &a, const String &b);
    friend int    operator>(const String &a, const String &b);

    void display() { cout << p << endl; }
};

s3 = s1 + s2;      // instead of strcat
if (s1 > s2) ...    // instead of strcmp
```

If you are asked to demonstrate string manipulation using operator overloading, this is the model answer. The two things the examiner is looking for are a **friend operator function** and a **destructor that releases what the constructor took**.

PREDICT THE OUTPUT

A binary member operator and a unary member operator in the same class.

```
#include <iostream>
using namespace std;

class Counter {
    int n;
public:
    Counter(int k = 0) { n = k; }
    Counter operator+(Counter c) { return Counter(n + c.n); }
    void operator-() { n = -n; }
    void show() { cout << n << endl; }
};

int main() {
    Counter a(7), b(3), c;
    c = a + b;
    c.show();
    -c;
    c.show();
    return 0;
}
```

Output

```
10
-10
```

`c = a + b;` is rewritten as `c = a.operator+(b);`. Inside, plain `n` is `a`'s 7 and `c.n` is `b`'s 3, so a new `Counter(10)` comes back and is assigned to `c`.

`-c;` is rewritten as `c.operator-();`. A unary member operator takes no argument at all, because its single operand is the calling object — so it flips `c`'s own `n` in place, and the second `show()` prints -10.

One extra thing to notice: `Counter(int k = 0)` is doing two jobs. It is the default constructor, which is how `Counter c;` compiles, and it is also a **converting constructor**, which is why `c = a + 2;` would work here — while `c = 2 + a;` would not, because `operator+` is a member.

CHAPTER 10 AT A GLANCE

- For a member operator, `a + b` becomes `a.operator+(b)`; for a friend it becomes `operator+(a, b)`.
- A binary operator takes **one** argument as a member and **two** as a friend; a unary takes **none** as a member and **one** as a friend.
- `2 + a` cannot work with a member operator, because an `int` has no member functions — a friend function fixes it.
- Cannot be overloaded at all: `::` `.` `*` `?::`. Must be a member: `=` `()` `[]` `->`.
- Postfix `++` is marked by a dummy `int` parameter that is never used.

CHAPTER 11

INHERITANCE AND THE THREE MODES

The inheritance deck is new material and is the densest thing on the paper — six to eight marks across two questions. This chapter settles the syntax, the reason protected exists, and the visibility table, which is the one table on this paper you should be able to draw from memory.

11.1 Why, and the is-a test

The word the lecture opens with is **reusability**. Write the common part once in a base class and derive from it, instead of writing the same three functions into `Bus`, `Car` and `Truck` and then fixing a bug in three places. Use inheritance only when the sentence "a derived **is a** base" is actually true: a car is a vehicle, a surgeon is a doctor.

THE COMMONEST DESIGN MISTAKE

"A car **has an** engine" is not an is-a relationship, so `Car` must not inherit from `Engine` — it should hold an `Engine` as a member. Getting is-a and has-a the wrong way round is asked about directly.

THE OLD CLASS	THE NEW CLASS
Base class	Derived class
Parent class	Child class
Super class	Sub class

11.2 The syntax

```
class derived_name : mode base_name
{
    // members of the derived class
};

class Car : public Vehicle { public: void horn(); };
```

THE DEFAULT NOBODY REMEMBERS

`class Car : Vehicle` and `class Car : private Vehicle` mean exactly the same thing. The class still compiles and its own member functions can use everything they inherited — but every public member of `Vehicle` is now private inside `Car`, so `main()` can no longer call any of them. For a `struct` the default is the opposite: public. Write the mode explicitly, every time.

11.3 Why protected exists

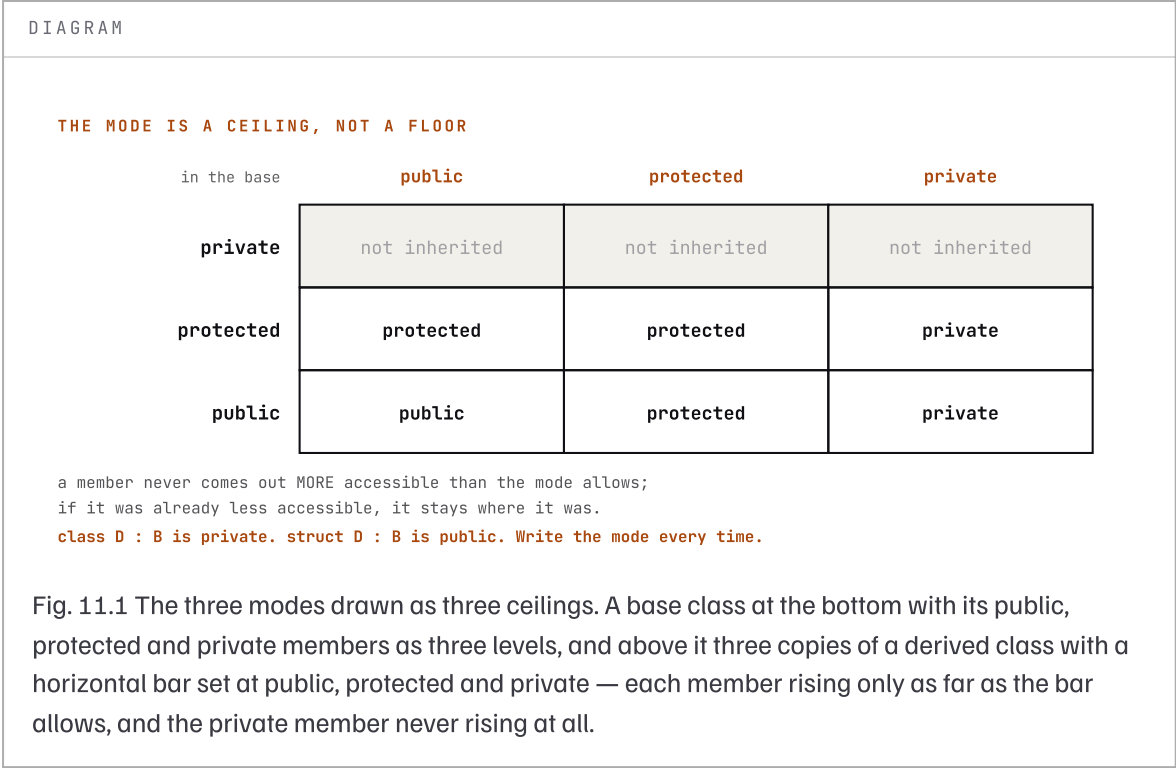
A private member of a base class is not accessible to the derived class at all. You could make it public so the derived class can reach it — but then the whole program can reach it and you have thrown away data hiding. `protected` is the third level that solves exactly this: **private to the family**.

```
class Base {
private:   int a;
protected: int b;
public:   int c;
        Base() { a = 1; b = 2; c = 3; }
        int getA() { return a; }      // the only route to a
};

class Derived : public Base {
public:
    void show() {
        // cout << a;          ERROR -- private, not accessible
        cout << b << endl;      // fine, protected
        cout << c << endl;      // fine, public
        cout << getA() << endl; // fine, through a public function
    }
};
```

11.4 The visibility table

MEMBER IN THE BASE	PUBLIC INHERITANCE	PROTECTED INHERITANCE	PRIVATE INHERITANCE
private	Not accessible	Not accessible	Not accessible
protected	protected	protected	private
public	public	protected	private



HOW TO REMEMBER IT IN ONE LINE

The mode is a **ceiling, not a floor**. Nothing comes out more accessible than the mode allows; anything already less accessible stays where it is. And the top row never changes: a private member of the base is not accessible in any derived class, in any mode.

Two things worth adding if there is room. A private member of the base is physically present in every derived object — it occupies space — but no derived class may name it. And a class that inherits privately can still **use** what it got; what it cannot do is pass it on, so anything derived from it gets nothing.

PREDICT THE OUTPUT

The same base class inherited two different ways. One call in `main` has been commented out — say why it had to be.

```
#include <iostream>
using namespace std;

class Base {
public:
    void hello() { cout << "hello" << endl; }
};

class Pub : public Base { };

class Pri : private Base {
public:
    void callIt() { hello(); }           // a member function -- allowed
};

int main() {
    Pub p;
    p.hello();                          // fine
    Pri q;
    // q.hello();                       // would not compile
    q.callIt();
    return 0;
}
```

Output

```
hello
hello
```

Inside `Pub`, `hello()` is still public, so an object of `Pub` may call it directly.

Inside `Pri`, private inheritance has pushed `hello()` down to private. `Pri`'s own member functions can still call it — private members are accessible inside their own class, which is why `callIt()` compiles — but an object of `Pri` cannot, so `q.hello()` is an error. The route out is through `Pri`'s own public function, and that is the second `hello`.

CHAPTER 11 AT A GLANCE

- Inheritance exists for **reusability**, and it is only the right tool when "derived is a base" is literally true.
- `class D : B` defaults to **private** inheritance, which usually breaks `main()` while leaving the class itself legal.
- protected means accessible in its own class and in derived classes, but not outside — private to the family.
- The visibility table: private is never accessible; protected stays protected except under private inheritance; public becomes public, protected or private to match the mode.
- The mode is a **ceiling** — nothing comes through more open than the mode, and nothing is ever made more accessible than it was.

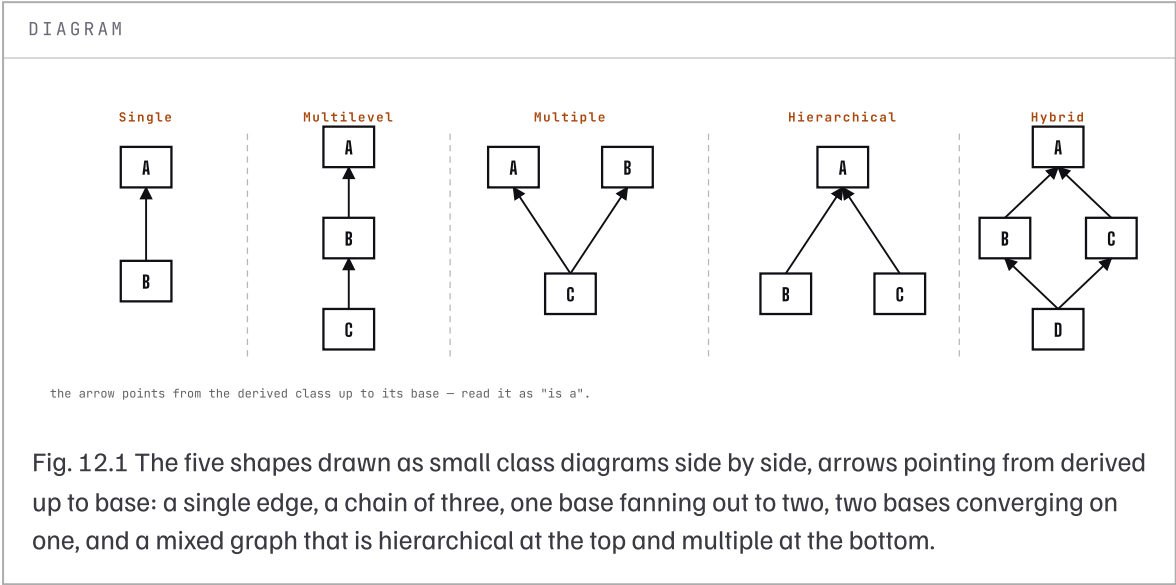
CHAPTER 12

SHAPES, AMBIGUITY AND THE VIRTUAL BASE

The second half of the inheritance question. This chapter settles the five shapes and their names, the order constructors run in when there is more than one base, the two completely different situations that both get called "ambiguity", and the diamond that the virtual base class exists to fix.

12.1 The five kinds

KIND	SHAPE	IN ONE LINE
Single	$A \rightarrow B$	One derived class, one base class.
Multilevel	$A \rightarrow B \rightarrow C$	A class derived from another derived class.
Hierarchical	$A \rightarrow B, A \rightarrow C$	Several derived classes from one base class.
Multiple	$A, B \rightarrow C$	One derived class from several base classes.
Hybrid	a mix	Two or more of the above combined — usually needs a virtual base.



```
class D : mode B1, mode B2, mode B3 { ... };

class Result : public Maths, public English { ... };    // correct
class Result : public Maths, English { ... };          // English is PRIVATE
```

TWO RULES OF THE MULTIPLE-INHERITANCE SYNTAX

The base classes are separated by commas, and the access mode must be written for **every one of them**. Writing `class D : public B1, B2` silently makes `B2` a private base, because the missing mode defaults to private.

12.2 Constructor order with several bases

THE RULE

Base class constructors run **first, in the order they are listed after the colon** — not alphabetical order, and not the order the classes were declared in. The derived class's own constructor runs last. Destructors run in exactly the reverse order.

12.3 Ambiguity: two situations, not one

The same word covers two different things, and the paper separates them.

```
// Situation 1 -- two bases with the same member name
class M { public: void display() { cout << "M" << endl; } };
class N { public: void display() { cout << "N" << endl; } };
class P : public M, public N { };

P p;
p.display();           // ERROR -- request for member display is ambiguous
p.M::display();        // fix -- prints M
p.N::display();        // fix -- prints N
```

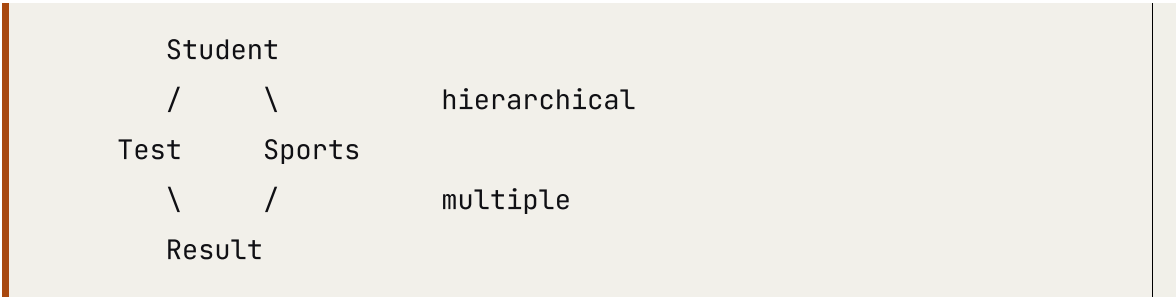
Note that class `P` itself compiles perfectly; the error appears only at the call. The better fix is to resolve it once inside `P`, so that users of `P` never see the problem: give `P` its own `display()` whose body calls `M::display();` .

```
// Situation 2 -- same name in base and derived: NO error
class A { public: void display() { cout << "From A" << endl; } };
class B : public A { public: void display() { cout << "From B" << endl; } };

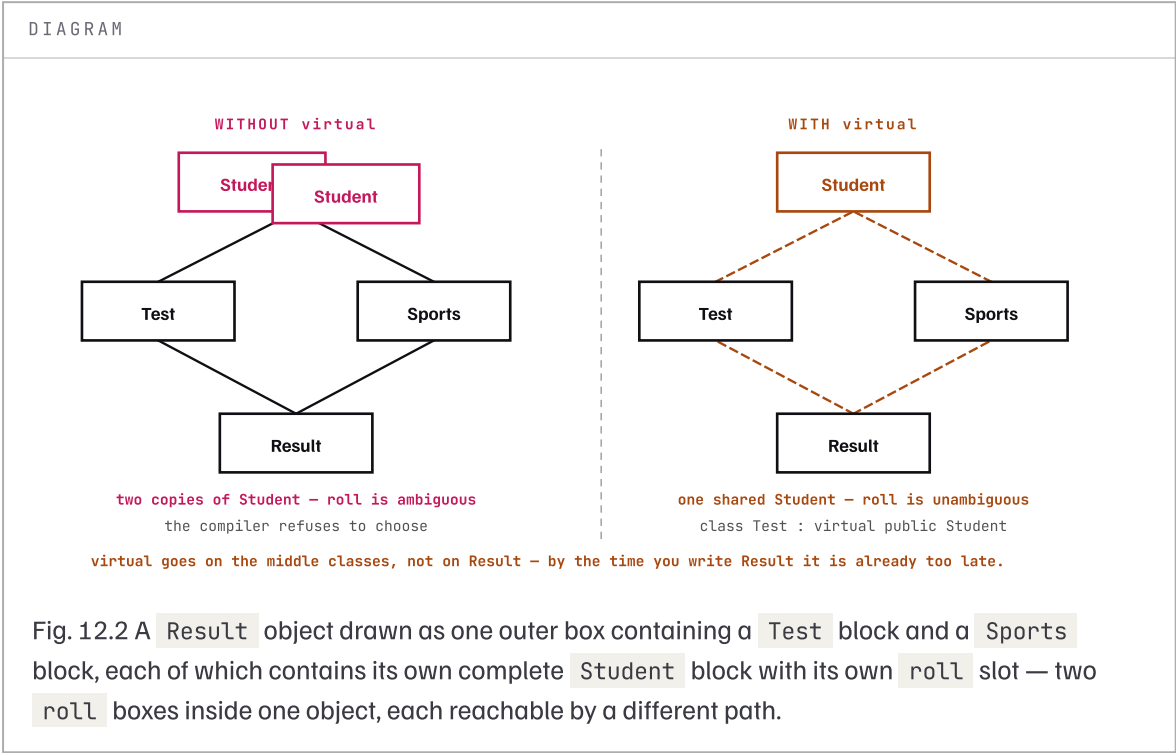
B b;
b.display();           // prints "From B" -- the nearer one hides the other
b.A::display();        // prints "From A" -- reached on purpose
```

	SAME NAME IN TWO BASES	SAME NAME IN BASE AND DERIVED
Shape	Side by side	One below the other
What the compiler does	Stops — neither is preferred	Nothing — the nearer one hides the other
A plain call goes to	Nowhere — it is an error	The derived version
How to reach the other	<code>obj.Base::func();</code>	<code>obj.Base::func();</code>

12.4 The diamond



`Test` inherits `Student`. `Sports` also inherits `Student`. `Result` inherits both, so a `Result` object physically contains **two copies** of everything `Student` owns. Now `roll` is ambiguous inside `Result` — and this time writing `Test::roll` would compile but be wrong in spirit, because a student has one roll number, not two.



12.5 The virtual base class

```
class Student { protected: int roll;
                public: void setRoll(int r) { roll = r; } };

class Test    : virtual public Student { protected: float marks; };
class Sports  : public virtual Student { protected: float score; };

class Result : public Test, public Sports {
public:
    void show() { cout << roll << endl; }    // fine -- exactly ONE roll now
};
```

DIAGRAM

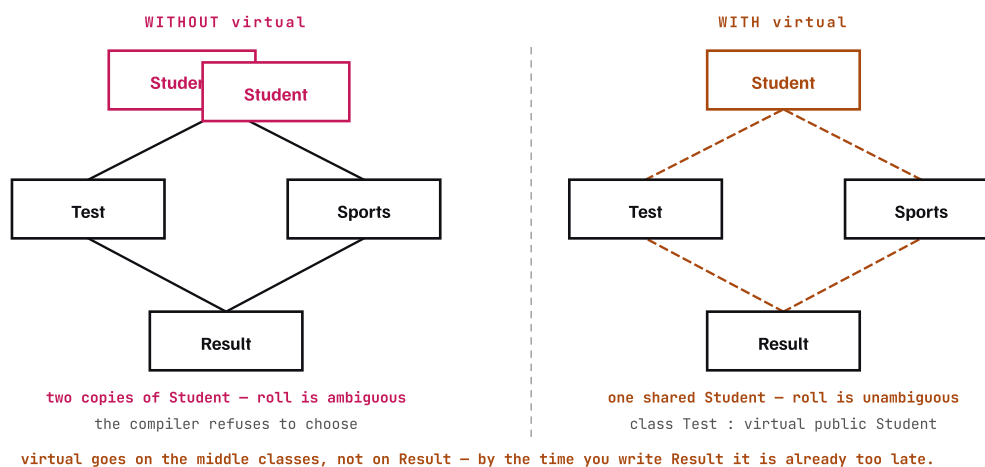


Fig. 12.3 The same object after `virtual`. One outer `Result` box with a `Test` block and a `Sports` block, and a **single** shared `Student` block off to one side with two arrows into it — one from each path.

- The keyword goes on the classes doing the **intermediate** inheriting — `Test` and `Sports` — not on `Result`. By the time you are writing `Result` it is too late.
- `virtual public` and `public virtual` are the same thing; the order of the two keywords does not matter.
- Remove the two `virtual` keywords and the program stops compiling, because `roll` inside `show()` becomes ambiguous again.
- With a virtual base, the **most derived class constructs it**. `Result`'s constructor calls `Student`'s directly, and the calls `Test` and `Sports` would normally make are skipped — that is the only way to build the shared sub-object exactly once.

TWO ONE-LINERS WORTH KEEPING

A **virtual base class** is not a **virtual function** — same keyword, unrelated jobs. And an **abstract class** is one with a pure virtual function, `virtual void f() = 0;`, which cannot be instantiated and exists only to be inherited from.

PREDICT THE OUTPUT

Six letters, in two lines. The base list is the whole question.

```
#include <iostream>
using namespace std;

class A { public: A() { cout << "A"; } ~A() { cout << "a"; } };
class B { public: B() { cout << "B"; } ~B() { cout << "b"; } };

class C : public B, public A {
public:
    C() { cout << "C"; }
    ~C() { cout << "c"; }
};

int main() {
    C obj;
    cout << endl;
    return 0;
}
```

Output

```
BAC
cab
```

Construction follows the order in the base list, which is **B** then **A** — not alphabetical order, and not the order the classes were declared in above. So the bases run first, **B** then **A**, and then **C**'s own constructor: **BAC**.

The `endl` ends that line while `obj` is still alive. Destruction is always the exact reverse of construction — **C**, then **A**, then **B** — and it happens when `obj` goes out of scope at the end of `main`, which is after the `endl`. So the second line is **cab**.

Swap the base list to `class C : public A, public B` and the output becomes **ABC** and **cba**.

CHAPTER 12 AT A GLANCE

- Five kinds: **single**, **multilevel**, **hierarchical**, **multiple**, **hybrid** — and the access mode must be written for every base in a multiple-inheritance list.
- Base constructors run first, **in the order listed after the colon**; destructors run in exactly the reverse order.
- The same name in **two bases** is a compile error at the call, fixed with `obj.Base::func();`.
- The same name in **base and derived** is not an error — the derived one hides the base one, and `obj.Base::func()` reaches the other.
- In the diamond, a `Result` object holds **two copies** of `Student`; declaring the `base virtual` in the two intermediate classes leaves exactly one.
- The `virtual` keyword goes on the intermediate classes, and the most derived class then constructs the shared base itself.

THE WHOLE SUBJECT IN ONE SCREEN

EVERYTHING, CONDENSED

01 PROCEDURAL VERSUS OBJECT ORIENTED

- The drawback of procedure oriented programming is that **global data is exposed** — any function can change it, so a bug anywhere can break anything.
- Object oriented programming locks the data inside the object and lets only that object's functions touch it.
- POP is top-down and organised around functions; OOP is bottom-up and organised around objects.
- The six core ideas: **class, object, encapsulation, abstraction, inheritance, polymorphism.**
- Encapsulation is the mechanism of hiding; abstraction is the purpose of hiding. That pair is asked constantly.

02 TOKENS, TYPES AND REFERENCES

- Five kinds of token: **keywords, identifiers, constants, strings, operators.** C++ has **63** keywords.
 - A leading zero means octal, so 037 is 31. 'A' is one byte; "A" is two, because of the hidden \0.
 - A struct is public by default, a class is private by default — that is the only real difference between them in C++.
 - In an enum, numbering continues from the last value you assigned, so {red, blue = 4, green} gives 0, 4, 5.
 - A **reference** is an alias, must be initialised at declaration, and can never be made to refer to a different variable afterwards.
-

03 THE OPERATORS C++ ADDS

- Unary `::name` reaches the **global** version of a name that a local declaration is hiding — something C cannot do at all.
- Binary `Class::member` names a member outside its class: member function definitions, static data definitions, static calls, inherited-name disambiguation.
- `new` calls the constructor and returns the right pointer type; `malloc` does neither. `delete` calls the destructor; `free` does not.
- Whatever you take with `new[]`, give back with `delete[]`. Using plain `delete` on an array is **undefined behaviour**.
- `setw` affects only the next item and needs `ios manip`; `endl` is the only manipulator that comes free with `iostream`.

04 OVERLOADING, INLINE AND DEFAULT ARGUMENTS

- Overloaded functions must differ in the **type, number or order** of parameters. A different return type alone is not an overload, it is an error.
- Resolution order: exact match, then promotion, then standard conversion, then user-defined conversion, then ellipsis — and **promotion beats conversion**.
- Default argument values belong in the **prototype**, and only trailing arguments may have them.
- A one-argument call that matches both an overload and a defaulted parameter is **ambiguous**, and the error appears at the call, not at the declaration.
- `inline` is a request the compiler may refuse; a member function defined inside the class body is inline automatically.

05 CALL BY VALUE, ADDRESS AND REFERENCE

- Call by value passes a **copy**, so the caller's variable can never change.
- Call by address passes the **address**; inside the function you must dereference with a star every time, and the call site shows an `&`.
- Call by reference passes an **alias**; the function body looks like call by value and so does the call, but the original does change.
- Call by reference is the only one of the three that C does not have.
- Use value for small unmodified types, reference to modify or to avoid copying a large object, address when a null argument has to be meaningful.

06 CLASSES AND OBJECTS

- A class declaration ends with `};` — the missing semicolon is the single most common syntax error in this subject.
 - Members of a class are **private by default**, and an object may reach them only through a public member function.
 - A member function defined outside the class needs the membership label `ClassName::`; without it, it is a global function with no access to private data.
 - A member function defined **inside** the class body is inline automatically.
 - Data members get one copy **per object**; member functions get **one copy for the whole class**.
 - Privacy is per class, not per object — a member function may read the private data of any object of its own class.
-

07 STATIC MEMBERS, FRIENDS, CONST AND THIS

- A static data member is **one variable shared by the whole class**, and it must be defined once outside the class or the program will not link.
 - A static member function is called as `ClassName::f()`, has no `this`, and may touch only other static members.
 - A **friend function is not a member**: no dot, no `ClassName::`, no `this`, and every object it works on is passed as an argument.
 - Friendship is granted by the class, is not mutual, and is not inherited.
 - A `const` member function may not modify the object, and a `const` object can call nothing else.
 - `this` is the hidden pointer to the calling object; `return *this;` and `if (this != &other)` are the two places you write it.
-

08 CONSTRUCTORS AND DESTRUCTORS

- A constructor has the class's name, no return type, runs automatically, and belongs in the public section.
 - Four kinds: **default, parameterized, copy, dynamic**. Constructors can be overloaded; a destructor cannot.
 - Write any constructor of your own and the free default one disappears — `Box b;` then fails to compile.
 - A destructor is `~ClassName()`, takes no arguments, and releases whatever the constructor allocated.
 - Objects are destroyed in the **reverse order of creation**, and an object in an inner block dies at that block's closing brace.
-

09 THE COPY CONSTRUCTOR, SHALLOW AND DEEP

- The copy constructor runs when an object is **created** from another, and when an object is passed or returned **by value**.
- `X b = a;` is the copy constructor because `b` is being created; `b = a;` on a later line is assignment.
- The parameter must be a reference — `A(A)` would need a copy to build its own argument, forever.
- A compiler-supplied copy is **shallow**: it copies the pointer, so two objects end up sharing one buffer and deleting it twice.
- The **rule of three**: a class that owns memory through a raw pointer needs a destructor, a copy constructor and a copy assignment operator.

10 OPERATOR OVERLOADING, MEMBER OR FRIEND

- For a member operator, `a + b` becomes `a.operator+(b)`; for a friend it becomes `operator+(a, b)`.
- A binary operator takes **one** argument as a member and **two** as a friend; a unary takes **none** as a member and **one** as a friend.
- `2 + a` cannot work with a member operator, because an `int` has no member functions — a friend function fixes it.
- Cannot be overloaded at all: `:: . * ? ::`. Must be a member: `= ( ) [ ] ->`.
- Postfix `++` is marked by a dummy `int` parameter that is never used.

11 INHERITANCE AND THE THREE MODES

- Inheritance exists for **reusability**, and it is only the right tool when "derived **is a** base" is literally true.
- `class D : B` defaults to **private** inheritance, which usually breaks `main()` while leaving the class itself legal.
- protected means accessible in its own class and in derived classes, but not outside — private to the family.
- The visibility table: private is never accessible; protected stays protected except under private inheritance; public becomes public, protected or private to match the mode.
- The mode is a **ceiling** — nothing comes through more open than the mode, and nothing is ever made more accessible than it was.

12 SHAPES, AMBIGUITY AND THE VIRTUAL BASE

- Five kinds: **single**, **multilevel**, **hierarchical**, **multiple**, **hybrid** — and the access mode must be written for every base in a multiple-inheritance list.
 - Base constructors run first, **in the order listed after the colon**; destructors run in exactly the reverse order.
 - The same name in **two bases** is a compile error at the call, fixed with `obj.Base::func()`;
 - The same name in **base and derived** is not an error — the derived one hides the base one, and `obj.Base::func()` reaches the other.
 - In the diamond, a `Result` object holds **two copies** of `Student`; declaring the base `virtual` in the two intermediate classes leaves exactly one.
 - The `virtual` keyword goes on the intermediate classes, and the most derived class then constructs the shared base itself.
-

That is the entire revision set — 63 lines. If a line does not make sense, the chapter it came from is one tap away, and the **full course** is behind that.